

CS1CA Computer Systems Architecture

Information Representation-4

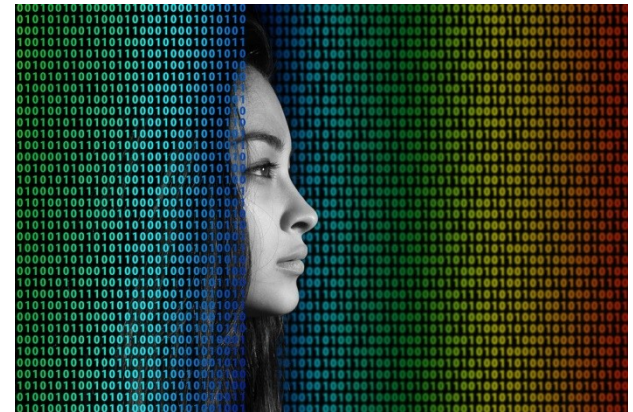
Week 2 lecture 1 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133 in Polly Vacher Building



Outline of the topic

- Introduction: what is information?
- Decimal, binary and hexadecimal number systems
 - Conversion between number systems
- Internal representation of characters
- Internal representation of integers
- Arithmetic and logic operations
- Internal representation of real numbers
 - IEEE 754 floating point format

In this week

- The following contents are delivered.
 - Arithmetic and logic operations in binary
 - Overflow issues
- The learning outcomes are listed below.
 - To be able to conduct arithmetic operations, such as addition, subtraction, multiplication, and division in binary, as well as shifting operations
 - To be able to conduct logic operations, such as AND, OR, XOR, and NOT
 - To be aware of overflow issues in fixed length integer representation/operation inside a computer system

Arithmetic/logic operations

- **Arithmetic operations:** add, subtract, multiply, divide
 - Precise action depends on how the values are encoded (two's complement versus floating-point).
- **Logic operations:** AND, OR, XOR
- **Shift operations:** circular shift, logical shift, arithmetic shift

Logic operations: AND, OR, and XOR

(recall Boolean algebra – bitwise operation)

The AND operation

$$\begin{array}{r} 0 \\ \text{AND } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ \text{AND } 1 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1 \\ \text{AND } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1 \\ \text{AND } 1 \\ \hline 1 \end{array}$$

The OR operation

$$\begin{array}{r} 0 \\ \text{OR } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ \text{OR } 1 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{OR } 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{OR } 1 \\ \hline 1 \end{array}$$

The XOR operation

$$\begin{array}{r} 0 \\ \text{XOR } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ \text{XOR } 1 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{XOR } 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{XOR } 1 \\ \hline 0 \end{array}$$

Logical Operations

AND	0	1	1	1	0	0	0	1
	1	0	1	1	1	1	0	0
	0	0	1	1	0	0	0	0

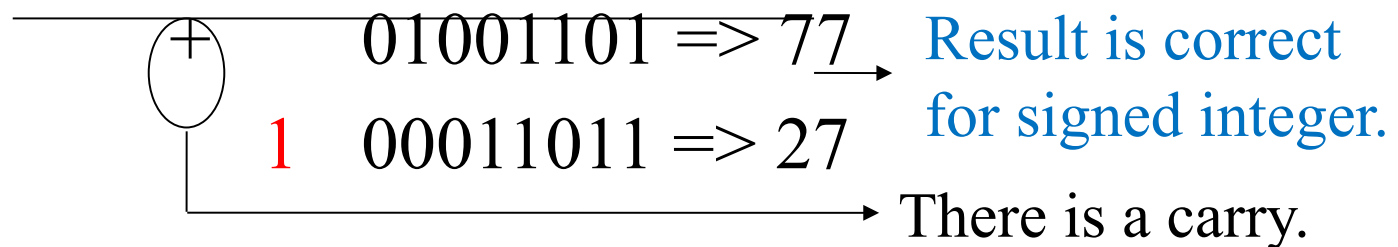
OR	0	1	1	1	0	0	0	1
	1	0	1	1	1	1	0	0
	1	1	1	1	1	1	0	1

XOR	0	1	1	1	0	0	0	1
	1	0	1	1	1	1	0	0
	1	1	0	0	1	1	0	1

Arithmetic operation: addition

- Binary addition is same as decimal addition.
- Example: 8-bit addition of two integers
 - Add 11001110 to 01001101

11001110 $\Rightarrow$ 206 (unsigned); -50
(signed)



– Data type matters!

Arithmetic operation: subtraction

- Binary subtraction is similar to binary addition, but works on two's complement.
- Example: $7-5=7+(-5)$ 7  0111; 5  0101
 - Two's complement of 0101 produces 1011  -5

$$\begin{array}{r} 0111 \\ + 1011 \\ \hline \bigcirc 1 0010 \text{  } 2 \end{array}$$

The carry is generated but the result is correct.

For signed integer, the carry is not used for result checking.

Shifting operations:

The LSB

- **Circular shift (rotation)**: a shift fills the hole by using the bit removed.
- **Logic shift**: a shift causes a hole in the end filled by 0.
- **Arithmetic shift**: shifts leave the sign bit unchanged.

A number: 1 1 1 0 0 1 0 1

with a right shift of 1 bit

Circular shift:

1 1 1 1 0 0 1 0 The bit is moved as the MSB.

Logic shift:

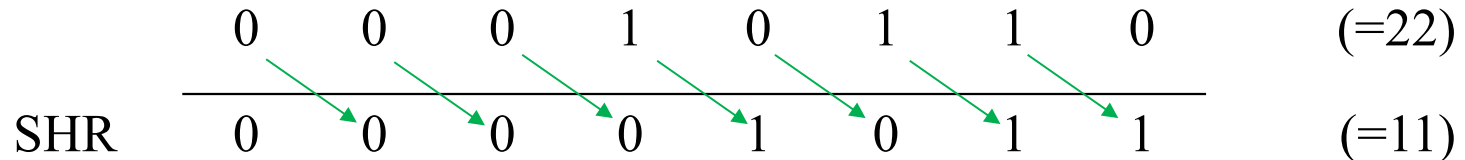
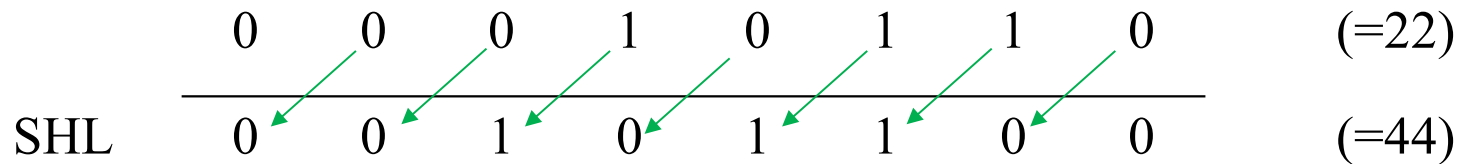
0 1 1 1 0 0 1 0 ← Lost the LSB

Arithmetic shift: in this case

1 1 1 1 0 0 1 0 ← Lost the LSB, but keep the sign bit unchanged.

Shift Operations

- Shift all bits to left (SHL) or right (SHR)



- Thus shifting left (SHL) is equivalent to multiplying by 2.
- Shifting right (SHR) is equivalent to dividing by 2

NB: “Shift right” 0000 0011 (=3) gets 0000 0001 (=1) not 1.5!

More Shift Operations

‘logical’ shift right (LSHR) vs ‘arithmetic’ shift right (ASHR)

	1	1	1	1	0	0	0	0	(=240)
LSHR	0	1	1	1	1	0	0	0	(=120)
	1	1	1	1	0	0	0	0	(=-16)
ASHR	1	1	1	1	1	0	0	0	(=-8)

- In ASHR, the most significant bit is unchanged.
- Thus the sign is unchanged. It is used in dividing signed numbers

Arithmetic operation:

multiplication and division

- Binary multiplication: similar to decimal operation – operate on each bit and add them together. Shifting binary string 1 bit to the left corresponds to multiplication by 2.
- Binary division: shifting a binary string 1 bit to the right corresponds to division by 2.
- The “sign bit” should be taken into account.

Think about algorithms for signed integer

Overflow: a problem of fixed length integers – addition

- 8-bit signed integers represent the range of
 $-128 \leq N_{ext} \leq 127$
- Consider the addition of two 8-bit integers (74+85) that gives a result greater than 127

External form

$$\begin{array}{r} 74 \\ + 85 \\ \hline 159 \end{array}$$

Internal form

$$\begin{array}{r} 01001010 \\ + 01010101 \\ \hline \textcircled{0} 10011111 \end{array}$$

- Internal form yields a binary result with the negative form;
- Adding two positive numbers should give a positive result, however it is out of the range, *i.e.* overflow has occurred.

Overflow: a problem of fixed length integers – subtraction

- Consider subtracting two 8-bit integers (-44-85) that give a result out of the range.

External form

- 44

- 85

- 129

Internal form

11010100

+ 10101011

0 1 01111111

- The carry as the Most Significant Bit (MSB) in the result has been suppressed by the 8 bits;
- A positive result is produced with a carry. -129 is out of the range, so that overflow occurs.

Overflow: a problem of fixed length integers – multiplication

- Consider multiplying two binary strings, one of length n and one of length m ;
- This will produce a product at most $n+m$ bits in length;
- Clearly there is a potential for overflow if the result is longer than the length allocated for integers by the architecture of the machine;
- True “32 bit” machines offer the facility for generating a 64 bit result from the two 32 bit operands in multiplication.

Overflow: a problem of fixed length integers – division

- Division is more complicated and again depends on the architecture of the machine;
- A true 32 bit machine will offer instructions that take two 32 bit operands (the dividend and divisor) and yield a 32 bit quotient and 32 bit remainder;
- However, there are “divide” instructions that allow a 64 bit dividend and a 32 bit divisor. The result must be 32 bit quotient and 32 bit remainder. This case has the potential for overflow, e.g. dividing any 64 bit number by one;
- Dividing by zero is a special case and when this occurs, an interrupt to the processor is generated.

Carry: problem of fixed length integers – 1

- Addition of two fixed length binary strings may generate a carry bit as the result.
- **Case 1:** adding a positive and a negative integer yielding a positive sum.

External form	Internal form
4 4	00101100
- 2 5	+ 11100111
1 9	00010011

1

Final carry bit set: other intermediate carries not indicated

Carry: problem of fixed length integers – 2

- Case 2: adding two negative integers always generates a carry.

External form		Internal form
- 1 0		11110110
- 2 5		+ 11100111
- 3 5		11011101
		

Final carry bit set: other intermediate carries not indicated

Processor architecture implementation

- All processors have a register (**status register**), which includes four named bits.

C	V	N	Z
---	---	---	---

occurred

C = 1 if carry occurred

V = 1 if overflow

N = 1 if result was

negative

Z = 1 if result was zero

- These bits are called the Condition Code (CC);
- The CC acts as a monitor for most **instructions**;
- For example, **Conditional Branch Instructions** test CC to implement a selection mechanism.

End of this lecture

Self-assessed Exercises – 1

- Complete the following calculations (8-bits)
 - $0x34 \text{ AND } 0xFF$; $0x76 \text{ OR } 0x9A$
 - $0x80 \text{ XOR } 0x00$; $0x33 \text{ XOR } 0x00$
 - $0x55 + 0xAA$, indicating the Condition Code (CC)
 - $0xA3 - 0xB6$, indicating the CC
 - Arithmetic right shift $0xA8$ one bit
 - Logic left shift $0xA8$ one bit
 - Circular right shift (rotate) $0xA8$ one bit

Note: 0x here indicates that the number is in hexadecimal.

End of this lecture

Self-assessed Exercises – 2

- **Convert** the following decimal numbers to 8-bit binary and **conduct** the operations. Find out how the results will set the condition code (i.e. C, V, N, Z) in the status register in an 8-bit signed integer representation. **You may verify your results in the lab practicals.**
 - $-39+14$
 - $-111+59$
 - $-39-52$
 - $108-108$

Please place your questions on the Discussion Board in BB.
We can discuss them in the tutorial session.